COPILOTTRAINING
TECHNICAL BRIEFING

Copilot Configuration Primitives

Instructions · Custom Prompts · Skills · Agents

Encode the convention once. Graduate the workflow when it is proven.

GitHub Copilot · Customization & Context

🤔 THE CORE QUESTION

"How can I make GitHub Copilot actually understand my codebase?"

Generic responses miss conventions that the repository never made explicit. Shared, reviewable configuration supplies that context.

Choose the smallest primitive, show its anatomy, then prove delivery and acceptance…

Individual contributors

Create repeatable commands and capabilities without rebuilding context in every session.

Team leads

Review shared behavior as version-controlled institutional knowledge with a named owner.

Platform engineers

Bound selectors, tools, and approval so configuration can scale across teams.

Encode

Turn one convention into repository, path-scoped, or directory guidance.

Graduate

Move a solved chat into a shared command, then a runnable skill.

Prove

Trace eligibility through loading, execution, validation, and acceptance.

📖 TABLE OF CONTENTS

PART 1

Instructions

Always-on and scoped guidance with observable delivery

PART 2

Custom Prompts

Freeze a solved workflow as a team /command

PART 3

Skills

Graduate the prompt with scripts and templates

PART 4

Agents

Compose the stack behind explicit tool boundaries

PART 1

Instructions

Place the context, prove delivery, then encode the shared baseline.



Place

Classify owner, lifetime, selector, and evidence.



Prove

Trace eligibility through accountable acceptance.



Encode

Use repository, path-scoped, or directory guidance.

```
$ /init
↳ Draft the baseline, inspect References, then validate the result.
```

Every Context Item Carries Four Decisions



Owner

Name who maintains, reviews, and may approve the context.



Lifetime

Choose one request, a repeated task, repository lifetime, or a policy cycle.



Selector

Target the repository, matching files, a command, a relevant task, or an agent.



Evidence

Define the loaded signal, executable check, artifact review, and acceptance owner.

🔐 One-request facts stay in the request. Shared behavior earns a reviewed primitive.

Trace Configuration from Eligibility to Acceptance

Each rung proves one claim and hands evidence to the next

1	2	3	4	TARGET 5
Eligible	**Loaded**	**Executed**	**Validated**	**Accepted**
The selector matches the request	The client included the artifact	Expected files and tools were used	Independent checks passed	The accountable reviewer approved
🎯	📥	🛠️	🧪	✅

References proves delivery. Tests and review prove the result.

A Valid Rule Can Miss the Request

🎯

1 · Name the target

The request edits src/orders/create-order.ts.

❌

2 · Find the break

applyTo targets src/payments/**/*.ts, so the valid rule is ineligible.

🔧

3 · Repair the selector

Change applyTo to src/orders/**/*.ts while preserving the rule.

✅

4 · Prove the result

Confirm loading, run Orders checks, inspect the diff, and obtain owner approval.

🎯 Inspect in order: target → selector → loaded context → tools → checks → review.

Three Instruction Surfaces — One for Each Scope

Repo-wide

The whole-repository baseline — always active.

- .github/copilot-instructions.md
- Applied to every request in this repo
- Coding standards, build commands, file structure
- No frontmatter needed — pure Markdown

Path-scoped

Activates only for matching file patterns.

- .github/instructions/*.instructions.md
- applyTo glob in frontmatter — required
- No applyTo means it is not auto-applied
- Language rules, tests, framework patterns

AGENTS.md

Portable agent playbook — open format.

- Root AGENTS.md is the default playbook
- Setup, test, and PR commands live here
- Cross-agent portable — not VS Code only
- Nested files are still experimental

A Production-Ready copilot-instructions.md

.github/copilot-instructions.md

```
# Repository Instructions

This repository uses TypeScript with strict type checking.

## Build and Test
- Build: `npm run build`
- Tests co-located in `__tests__/` directories
- Use Vitest — never Mocha or Jest
- Run: `npm test`

## Coding Standards
- Prefer functional patterns
- Explicit return types on all functions
- JSDoc on every exported function
- Named exports only — no default exports

## Error Handling
- Custom error classes extending Error
- Structured logging via logger.error()
- Never swallow errors silently
```

Immediate effect

Created this file? Every eligible Copilot request in this repo now receives your conventions.

Keep it concise

Preserve context capacity for the request, relevant code, and tool results.

Verify via References

After the first chat response, check the References panel in VS Code — instruction files used are listed there.

Path-Scoped Instructions — Precision at File Level

.github/instructions/models.instructions.md

```
---
applyTo: "src/models/**/*.ts"
---

# Database Model Instructions

When working with database models:

1. Use Prisma schema in `prisma/schema.prisma`
2. Include JSDoc with field descriptions
3. Define relationships with `@relation`
4. Add indexes for foreign keys
5. snake_case for DB columns, camelCase in TS
6. Always include: `createdAt` and `updatedAt`
```

🎯 Surgical scoping

applyTo glob means these rules activate only on matching files — never polluting other contexts.

⚠️ No glob, no auto-load

Omit applyTo and the file is not applied automatically. You can still attach it by hand.

➕ Additive with repo-wide

Path-scoped files combine with copilot-instructions.md when both selectors match.

Write Instructions Like They Cost Tokens

1

One idea per line

Each rule is a single statement. Split compound guidance into separate lines.

💡

Include the why

Use date-fns, not moment — moment is deprecated and inflates the bundle.

✅

Show examples

Preferred vs avoided snippets beat abstract style adjectives every time.

🧹

Skip the linter

Do not spend context on tabs, quotes, or import order. Tools already own that.

AGENTS.md — Cross-Tool Institutional Knowledge

What goes in root AGENTS.md

- Setup and install commands for the repo
- Test commands and linting steps for PRs
- Repository navigation tips
- PR title format and commit conventions
- Commands agents actually run

Keep the playbook portable

- Open Markdown — not VS Code-specific
- Any coding agent that reads AGENTS.md gets it
- Prefer applyTo for folder-specific rules
- Nested AGENTS.md is still experimental
- Institutional knowledge that travels with the repo

Demo: /init Drafts the Baseline

Do not hand-author the first constitution. Generate it, then edit.

```
> "/init"

🤔 Copilot:

  Scanning workspace conventions…
  Found: TypeScript, Vitest, Prisma, pnpm
  Drafting .github/copilot-instructions.md

✅ Always-on file created — review, trim, commit

> "Add a user endpoint"

🤔 Copilot:

  References: copilot-instructions.md
  Prisma + Vitest + named exports — not generic Express

✅ Check References. If the file is listed, it is working.
```

5 minutes to a team-aware baseline

PART 2

Custom Prompts

Solve the task in chat. Freeze the working recipe as a team /command.

🧪

Solve first

A prompt is a proven workflow — not a guess about what the team might need later.

👥

Team /command

One developer writes the file. Every developer gets /component with the same context.

🎓

Then graduate

When the recipe needs scripts or auto-load, it becomes a skill. Not before.

```
$ /create-prompt scaffold a React component with tests
↳ Freeze the recipe. Do not retype it tomorrow.
```

Prompt File Anatomy

.github/prompts/component.prompt.md

```
---
name: component
description: Generate a React component with TypeScript,
  tests, and documentation
tools: ["editFiles", "createFile"]
agent: agent
---

# Component Generator

Create a new React component: ${input:componentName:Component

## Files to Create
src/components/${input:componentName}/
  ${input:componentName}.tsx
  ${input:componentName}.types.ts
  __tests__/${input:componentName}.test.tsx
  index.ts

Follow conventions in [standards](../copilot-instructions.md)
Also available: ${selection} and ${file}
```

Real variables

${input:name:hint} prompts the user. ${selection} and ${file} pull the current editor context.

Link, do not copy

Reference copilot-instructions.md. Duplicating the constitution creates drift.

Scope the tools

tools and agent on the prompt override the current session for this one task.

Worked Example: /create-prompt From a Solved Chat

Do not invent the file from a blank buffer. Extract the recipe that just worked.

```
> "Scaffold a React component with types, CSS modules, and a Vitest file."

🤔 Copilot:

  Created Button.tsx, Button.types.ts, Button.test.tsx
  Used named exports and Vitest — from instructions

> "/create-prompt Save this as a reusable /component command"

🤔 Copilot:

  Writing .github/prompts/component.prompt.md
  Parameter: ${input:componentName}
  Linked ../copilot-instructions.md

✅ Team can now type /component — the recipe is frozen
```

One writer. Every developer gets the slash command.

Prompt First — Skill When the Recipe Is Proven

Stay on a prompt when

- A human decides when it runs
- The steps are text, not scripts
- You are still iterating on the recipe
- The team needs a shared /command now
- Example: /component, /pr-checklist

Graduate to a skill when

- The agent notices the task on its own
- You have a script or template to run
- The same workflow spans tools
- You want auto-load, slash-only, or both
- Example: test-runner with run-tests.sh

Keep Prompts Thin — They Are Not the Constitution

Do not copy rules

Link instructions. Copied standards drift the first week.

- **Link copilot-instructions.md**
- **One source of conventions**
- **Prompts describe the task only**

Do not skip variables

Hard-coded names make a personal snippet, not a team command.

- **${input:name:hint} for parameters**
- **${selection} for the highlight**
- **${file} for the active file**

Do not start here

A prompt without instructions still produces generic files.

- **Instructions first**
- **Then freeze the recipe**
- **Then add scripts as a skill**

PART 3

Skills

Graduate the proven prompt. Add scripts and templates the agent can actually run.

📦

A pack, not a file

SKILL.md plus scripts/ plus templates. If there is nothing to run, keep the prompt.

🎛️

Both by default

A skill can auto-load and appear as /skill. Lock either side on purpose.

🧪

Live /create-skill

Extract the prompt you just proved. Do not invent a capability from a blank folder.

```
$ /create-skill from how we just ran and fixed tests
↳ Prompt solved it. Skill packages it.
```

Three-Level Progressive Loading

1

Discovery

Lightweight metadata scanned at startup.

- Skill name and description only
- Always loaded — near-zero context cost
- Powers AI relevance matching
- Dozens of skills with minimal overhead

2

Instructions

Full SKILL.md body — when the task matches.

- Complete guidance and process steps
- Loaded only when description matches
- Or when someone types /test-runner
- Contains links to scripts and templates

3

Resources

Scripts and templates — when referenced.

- Shell scripts, test templates, examples
- Fetched lazily during execution
- Never loaded if not referenced
- This is why a skill is a folder

A Test-Runner Skill — The Folder Is the Point

.github/skills/test-runner/SKILL.md

```
---
name: test-runner
description: Run tests, analyze failures, and suggest
  fixes. Use when asked to test, debug failures,
  or add coverage.
---

# Test Runner Skill

## Process
1. Detect the runner from package.json
2. Run [scripts/run-tests.sh](scripts/run-tests.sh)
3. Start new files from [assets/test-template.ts](assets/test-
4. Suggest targeted fixes from the failure output
```

🎯 Description is the trigger

Write when to use it. That sentence is how auto-load decides.

🔗 Links make resources real

Unlinked files in the folder are invisible. Reference every script and template.

📁 .github/skills/test-runner/

SKILL.md + scripts/ + assets/. Personal copy lives in ~/.copilot/skills/.

The Script the Skill Actually Runs

.github/skills/test-runner/scripts/run-tests.sh

```
#!/usr/bin/env bash
set -euo pipefail

# scripts/run-tests.sh — invoked from SKILL.md
TARGET="${1:-}"

if [[ -n "$TARGET" ]]; then
  npm test -- "$TARGET"
else
  npm test
fi

if [[ "${2:-}" == "--coverage" ]]; then
  npm run test:coverage
fi
```

▶️ Runnable, not descriptive

The agent executes this script. A paragraph of advice is still a prompt.

🔗 Must be linked

SKILL.md has to point at scripts/run-tests.sh or the file never loads.

🔒 Keep it boring

Detect the runner, run the suite, print failures. Leave analysis to SKILL.md.

The Template New Tests Start From

.github/skills/test-runner/assets/test-template.ts

```
import { render, screen, waitFor } from "@testing-library/reac
import userEvent from "@testing-library/user-event";
import { ComponentUnderTest } from "./ComponentUnderTest";

describe("ComponentUnderTest", () => {
  it("renders without crashing", () => {
    render(<ComponentUnderTest />);
    expect(screen.getByRole("main")).toBeInTheDocument();
  });
});
```

Start from house style

New tests copy this file. The agent does not invent a second test stack.

Linked from SKILL.md

assets/test-template.ts is loaded only when the skill says to create a test.

Why this is a skill

A prompt can describe a test. A skill ships the template and the runner.

Demo: /create-skill From the Prompt That Worked

Graduate after the recipe is proven. Then add the script and the template.

```
> "/create-skill from how we just ran and fixed the API tests"

🤔 Copilot:

  Creating .github/skills/test-runner/
  SKILL.md + scripts/run-tests.sh + assets/test-template.ts

> "Make it auto-load on test work, and keep /test-runner"

🤔 Copilot:

  Default: auto-load AND slash command
  user-invocable: false hides /skill
  disable-model-invocation: true is slash-only

✅ Ask to test an endpoint — the skill loads and runs the script
```

Prompt solved it. Skill packages the runner.

PART 4

Agents

Compose the stack into a persona. The tool list is the architecture.

🔒

Constraint as design

A read-only Planner cannot edit files. The restriction is the design, not a workaround.

🔄

Handoff, not hop

Plan, then one button to Implementer with the plan intact. Role stays separated.

🧪

Live /create-agent

Generate the persona last — after instructions, prompts, and skills already exist.

```
$ /create-agent a read-only planner that hands off to implementer
↳ Do not start teams here.
```

The Planner Agent — Constraint as Architecture

.github/agents/planner.agent.md

```
---
name: planner
description: Generate implementation plans by researching
  the codebase. Read-only — never modifies files.
tools: ["search", "readFile", "listFiles", "fetch"]
model: Claude Sonnet 4 (copilot)
handoffs:
  - label: Start Implementation
    agent: agent
    prompt: Implement the plan outlined above.
    send: false
---

# Planning Agent

You are a senior architect creating implementation plans.

## Rules
- NEVER modify files — you are read-only
- Always cite files and line numbers
- Include effort estimates for each step
```

🔒 tools: is the boundary

search, readFile, listFiles, fetch — no editFiles. Same pattern as a security reviewer.

🔄 Handoff button appears

Start Implementation switches to the implementer with the plan intact.

🧠 Model per agent

Reasoning model for planning. Fast model for implementation. Match the role.

Worked Example: /create-agent After the Stack Exists

Generate the persona after the constitution, the prompt, and the skill already exist.

```
> "/create-agent a read-only planner that hands off to implementer"

🤔 Copilot:

  Writing .github/agents/planner.agent.md
  tools: search, readFile, listFiles, fetch
  handoff: Start Implementation → agent

> "Plan adding rate limiting to the users API"

🤔 Copilot:

  Read src/api/users.ts:88-140
  Plan: middleware + tests via test-runner skill
  No files edited — Planner has no write tools

✅ Handoff button appears. Constraint held.
```

Do not start teams on agents.

The Configuration Stack — How the Primitives Compose

S1

Instructions

Always-on baseline every later file inherits

S2

Prompts

Freeze a solved workflow as a team /command

S3

Skills

Graduate the recipe — add scripts and templates

TARGET

S4

Agents

Constrain the persona that composes the rest

Solve with a prompt. Graduate to a skill. Constrain with an agent.

Choosing the Right Primitive

Always-on	Repo conventions and build commands belong in every request.	→ Instruct
Folder rules	applyTo for file patterns. Root AGENTS.md for portable commands.	→ Scope
Solved task	The team repeats a proven recipe and shares one /command.	→ Prompt
Add scripts	The recipe needs a runner, a template, or auto-load.	→ Skill
A persona	A role needs tools on or off, and maybe a handoff.	→ Agent

THE VALUE RECAP

From Repeated Context to Inherited Knowledge

Before

- Developer re-explains conventions in every chat session — minutes wasted, every day
- The same scaffolding prompt is retyped until someone copies the wrong version
- A skill folder is invented before the team has a working recipe
- Generic boilerplate: no types, console.log errors, wrong test framework

After

- /init drafts copilot-instructions.md — conventions present on the next request
- /create-prompt freezes the recipe the team just proved as /component
- /create-skill adds run-tests.sh and the test template only after that recipe works
- /create-agent last — a read-only Planner that cannot touch the files it planned

5 min
To /init the first instructions file and see immediate improvement

Prompt → Skill
Solve thoroughly as a /command, then graduate scripts into a skill

4 primitives
Instructions, prompts, skills, agents — in that order

✅ WHAT YOU CAN DO TODAY

Today

- Run /init and commit a trimmed .github/copilot-instructions.md
- Verify the References panel lists that file on the next chat request
- Write one rule with a why and a preferred vs avoided example

This Week

- Add one path-scoped .instructions.md — and remember applyTo is required
- Run /create-prompt on a scaffolding task your team already solved
- Link that prompt to instructions instead of copying the constitution

This Month

- Run /create-skill only after the prompt is stable — add a script and a template
- Set user-invocable or disable-model-invocation on purpose, not by accident
- Run /create-agent for a read-only Planner and try the handoff once

🔑 Key Takeaway

Start with instructions. Solve with a prompt. Graduate to a skill. Constrain with an agent.

REFERENCES

VS Code Documentation

Customize AI in Visual Studio Code
Overview of all customization options — the canonical starting point

Use custom instructions in VS Code
Instructions syntax, applyTo globs, and personal vs. workspace scoping

Use Agent Skills in VS Code
Skill structure, progressive loading, and the open standard

Use prompt files in VS Code
Prompt frontmatter schema, variable interpolation, and team sharing

Custom agents in VS Code
Agent file structure, tool restrictions, model selection, and handoffs

Chat Debug View
Inspect request context, tool calls, and response details

GitHub Documentation

Adding repository custom instructions for GitHub Copilot
GitHub-side documentation for repository instructions

AGENTS.md open format
Open specification for the cross-agent AGENTS.md playbook format



Copilot Configuration Primitives

ENCODE THE CONVENTION ONCE. GRADUATE THE WORKFLOW WHEN IT IS PROVEN.



Instructions

/init today — one file, immediate improvement, zero ongoing maintenance



Prompts

Freeze a solved recipe as a team /command before you automate it



Skills

Graduate the prompt — scripts and templates the agent can run

Solve with a prompt. Graduate to a skill. Add an agent only when the role needs a tool boundary.